

Classi e interfacce

class: definibili solo metodi concreti
static class: definibile solo internamente a un'altra classe e utilizzabile senza istanziare un oggetto della classe esterna
final class: non estendibile con sottoclassi
abstract class: definibili intestazioni (pubbliche) di metodi astratti e/o metodi concreti (pubblici e privati)
interface: definibili solo intestazioni di metodi pubblici

Lettura e stampa

```
try(Scanner sc = new Scanner(System.in))          System.out.println(String what)
    sc.nextInt();
```

Caratteri e stringhe

```
char charAt(int index)                          String substring(int start, int end)
int compareTo(String other)                     String toLowerCase()
int compareToIgnoreCase(String other)           String toUpperCase()
boolean equals(Object other)                   String trim()
String String.format(String format, Object args) String isEmpty()
int indexOf(String what)                       String isBlank()
```

Controllare se una stringa `s` è composta da soli caratteri alfabetici: `s.chars().allMatch(Character::isAlphabetic)`

Controllare se una stringa `s` è composta da soli caratteri minuscoli: `s.chars().allMatch(Character::isLowerCase)`

Formattare una stringa `s` con numero di caratteri prefissato e allineamento a destra: `String.format("%10s", s)`

Formattare un intero `i` con un numero di cifre prefissato: `String.format("%05d", i)`

Formattare un decimale `f` con un numero di cifre prefissato: `String.format("%.2f", f)`

Random

```
Random random = new Random()                    double nextDouble()
int nextInt(int min, int max)                   long nextLong()
boolean nextBoolean()                          void setSeed(long seed)
```

Enumerazioni

```
enum Name {                                     E[] values()
    FIRST,                                     E valueOf(String name)
    SECOND,                                   int compareTo(E other)
    ...                                       int ordinal()
}
```

Array

```
E[] Arrays.sort(E[] arr)                      List<T> Arrays.asList(T[] arr)
E[] Arrays.sort(E[] arr, Comparator<E> c)    Stream<T> Arrays.stream(T[] arr)
E[] Arrays.fill(E[] arr, E val)               int Arrays.hashCode(T[] arr)
boolean Arrays.equals(E[] arr1, E[] arr2)     String Arrays.toString(T[] arr)
```

Collezioni

```
boolean add(E element)                        int size()
boolean addAll(Collection<E> other)           T[] toArray(new T[] )
boolean contains(Object element)              Stream stream()
boolean containsAll(Collection<E> other)      T Collections.binarySearch(List<T> list, T key)
boolean equals(Collection<E> other)           Collection Collections.sort(List<T> list)
boolean isEmpty()                            void Collections.copy(List<T> dest, List<T> src)
boolean remove(Object element)               int Collections.frequency(Collection<E> c, Obj o)
boolean removeAll(Collection<E> other)        T Collections.min(Collection<E> c)
boolean retainAll(Collection<E> other)        T Collections.max(Collection<E> c)
```

Liste

Implementazioni: ArrayList, LinkedList, Vector, Stack

```
ArrayList(Collection<? extends E> parent)    int indexOf(Object element)
LinkedList(Collection<? extends E> parent)   boolean remove(Object element)
boolean add(E element)                       E remove(int index)
void add(int index, E element)               E set(int index, E element)
E get(int index)                             List<E> List.of(E e1, E e2, ...)
```

Insiemi

Implementazioni: `HashSet` (elementi non ordinati), `TreeSet` (elementi ordinati)

<code>HashSet(Collection<? extends E> parent)</code>	<code>E first()</code> (solo <code>TreeSet</code>)
<code>TreeSet(Collection<? extends E> parent)</code>	<code>E last()</code> (solo <code>TreeSet</code>)

Mappe

Implementazioni: `HashMap` (chiavi non ordinate), `TreeMap` (chiavi ordinate)

<code>HashMap(Map<? extends K, ? extends V> parent)</code>	<code>K getKey()</code>
<code>TreeMap(Map<? extends K, ? extends V> parent)</code>	<code>V getValue()</code>
<code>boolean containsKey(Object key)</code>	<code>Collection<V> values()</code>
<code>boolean containsValue(Object value)</code>	<code>V put(K key, V value)</code>
<code>V get(Object key)</code>	<code>V putIfAbsent(K key, V value)</code>
<code>V getOrDefault(Object key, V defaultValue)</code>	<code>V replace(K key, V value)</code>
<code>boolean isEmpty()</code>	<code>V remove(Object key)</code>
<code>Set<K> keySet()</code>	<code>K firstKey()</code> (solo <code>TreeMap</code>)
<code>Set<Map.Entry<K, V>> entrySet()</code>	<code>K lastKey()</code> (solo <code>TreeMap</code>)

Istanziare un oggetto (valore) e ottenerlo se la chiave è assente, oppure ottenere il valore inserito se la chiave è presente:

```
ThisClass value = map.computeIfAbsent(key, k -> new ThisClass());
```

Confronti tra oggetti

Rendere una classe comparabile: `ThisClass implements Comparable<ThisClass> { public int compareTo... }`

Specificare un ordinamento diverso da quello stabilito dal `compareTo` alle strutture dati ordinate:

<pre>Set s = new TreeSet<Class>(new Comparator<Class>(){ Oppure: public int compare(Class o1, Class o2) { int diff = o1.field1 - o2.field1; if(diff != 0) return diff; ... return o1.fieldN.compareTo(o2.fieldN); } });</pre>	<pre>Set s = new TreeSet<Class>(this::compare); private int compare(Class o1, Class o2) { ... } Oppure: Set s = new TreeSet<Class>((o1, o2) -> { ... });</pre>
---	--

Gestione dei file

Lettura da file testuale:

```
try(BufferedReader reader =
    new BufferedReader(new FileReader(fileName))) {
    String line;

    while((line = reader.readLine()) != null) {
        ...
    }
}
```

Oppure:

```
try(Scanner reader =
    new Scanner(new FileReader(fileName))) {
    while(reader.hasNextLine()) {
        String line = reader.nextLine();

        ...
    }
}
```

Scrittura su file testuale:

```
try(BufferedWriter writer =
    new BufferedWriter(new PrintWriter(fileName))) {
    writer.println("...");
    writer.printf("...", arg1, arg2, ..., argN);
}
```

Lettura e scrittura da file binario:

```
try (InputStream reader =
    new BufferedInputStream(
        new FileInputStream("..."));
    OutputStream writer =
    new BufferedOutputStream(
        new FileOutputStream("..."))) {
    int c;

    while ((c = reader.read()) != -1) {
        writer.write(c);
    }
}
```

Iterazioni sugli oggetti

Per ottenere gli elementi di una struttura interna iterando sull'oggetto:

```
class ThisClass implements Iterable<ElementClass> {
    Collection<ElementClass> c = new Collection<ElementClass>();

    public Iterator<ElementClass> iterator() {
        return c.iterator();
    }
}
```

Thread

```
Thread()
Thread(Runnable r)
void start()
void run()
```

```
void join() throws java.lang.InterruptedException
void Thread.sleep(long milliseconds)
Thread Thread.currentThread()
```

Interfacce funzionali

```
Consumer<T> void accept(T t)
Predicate<T> boolean test(T t)
```

```
Supplier<T> T get()
Function<T, U> boolean U apply(T t)
```

Lambda espressioni

Metodo di una classe come espressione a singolo parametro:

```
obj.method(p -> Class.method(p))
obj.method(p -> p.method())
obj.method(Class::method)
```

Metodo locale come espressione a singolo parametro:

```
obj.method(p -> method(p))
obj.method(this::method)
```

Stream

```
IntStream Intstream.range(int a, int b)
IntStream Intstream.rangeClosed(int a, int b)
IntStream Intstream.iterate(int s, Function f)
Stream<T> filter(Predicate p)
Stream<T> map(Function f)
Stream<T> limit(long max)
T max()
T min()
T findFirst()
Stream<T> distinct()
long count()
```

```
void forEach(Consumer action)
boolean noneMatch(Predicate predicate)
boolean allMatch(Predicate predicate)
boolean anyMatch(Predicate predicate)
Stream<T> sorted()
int sum()
int average()
T orElse(T else)
T[] toArray()
List<T> toList()
Stream<T> parallel()
```

Metodi comuni da definire

```
public int compareTo(ThisClass other) {
    int diff = field1 - other.field1;
    if (diff != 0) return diff;

    diff = field2 - other.field2;
    if (diff != 0) return diff;

    ...

    return fieldN.compareTo(other.fieldN);
}

public boolean equals(Object other) {
    return other instanceof ThisClass oAsTC &&
        this.field1 == oAsTC.field1 &&
        this.field2 == oAsTC.field2 &&
        ... &&
        this.fieldN.equals(oAsTC.fieldN);
}

public int hashCode() {
    return field1.hashCode() ^
        field2.hashCode() ^
        ... ^
        fieldN.hashCode();
}
```

```
public Iterator<ThisClass> iterator() {
    return new Iterator<ThisClass>() {
        public boolean hasNext() {
            ...
        }

        public ThisClass next() {
            ...
        }
    };
}

public E mostFrequent() {
    Map<E, Integer> frequencies = new HashMap<>();

    for (E e : collection) {
        int old = frequencies.getOrDefault(e, 0);
        frequencies.put(e, old + 1);
    }

    E result = null;
    int maxFrequency = 0;

    for (E e : collection) {
        int frequency = frequencies.get(e);
        if (result == null ||
            frequency > maxFrequency) {
            result = e;
            maxFrequency = frequency;
        }
    }

    return result;
}
```